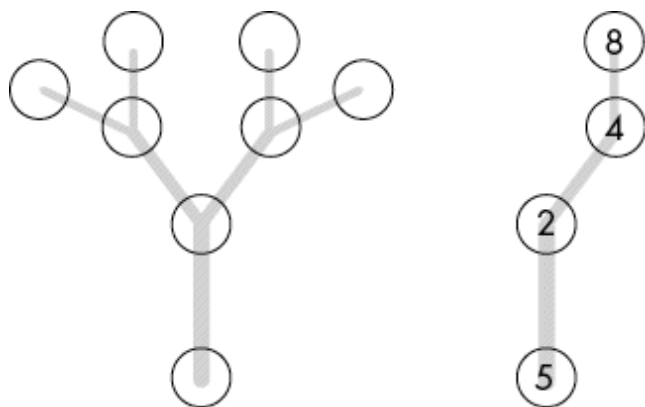


Recall from Chapter 2 that recursion is especially suited for problems that involve a tree-like structure and backtracking. An array, string, or other linear data structure can be considered a tree-like structure, albeit a tree that has only one branch at each node, as in [Figure 3-2](#).



*Figure 3-2: A [5, 2, 4, 8] array (right) is like a tree data structure (left) with only one branch at each node.*

The key “tell” that our recursive function is unnecessary is that it never does any backtracking over the data it processes. It makes a single pass over each element in the array from beginning to end, which is something a basic loop can accomplish. Additionally, the Python recursive summation function is about 100 times slower than a straightforward iterative algorithm. Even if performance weren’t an issue, the recursive `sum()` function would cause a stack overflow if passed a list with tens of thousands of numbers to sum. Recursion is an advanced technique, but it isn’t always the best approach.

In Chapter 5, we’ll examine a recursive summation function that uses a divide-and-conquer strategy, and in Chapter 8 we’ll examine one that uses tail call optimization. These alternate recursive approaches work around some of the problems in the summation function in this chapter.